



EMBEDDED SOFTWARE ENGINEERING

Session 1 – C Refresher

Laurent Fiack – laurent.fiack@ensea.fr

Bureau D212

Today

- 15-minute C quiz (individually, no notes)
- Then: a fast tour of the C you already know – no new syntax today
- Goal: everyone starts session 2 with the same baseline
- This is a review, not a first lecture on C: expect speed

Today

- 1 Warm-up
- 2 Variables & Types
- 3 Memory Model
- 4 Pointers
- 5 Arrays & Strings
- 6 Composite Types
- 7 Dynamic Allocation
- 8 Preprocessor
- 9 Compilation & Linking
- 10 Project Organization

Warm-up

Welcome back from the summer – two minutes to reboot

```
int is_valid(int x) {  
    if (x < 0 || x > 100) {  
        return 0;  
    }  
    return 1;  
}
```

```
uint8_t mask = flags & (1 << 3);    // read bit 3  
flags |= (1 << 2);                  // set bit 2  
flags &= ~(1 << 1);                 // clear bit 1  
flags ^= (1 << 0);                  // toggle bit 0
```

- Variables, if/else, loops, functions: assumed known
- Bitwise operators (& | ^ ~ << >>): you will use these constantly on registers

Fixed-width types: `stdint.h`

```
#include <stdint.h>

uint8_t  count;    // 0 .. 255
int16_t  offset;   // -32768 .. 32767
uint32_t timestamp_ms;
```

- `int`, `short`, `long`... have no guaranteed size
- On a microcontroller you care about exact width: register sizes, memory budget, overflow
- Always prefer `uintN_t` / `intN_t` for anything hardware-related

Scope and lifetime

Scope – where a name is visible

- Block scope (inside {})
- File scope (outside any function)

Lifetime – when storage exists

- Automatic: local variables, live on the stack
- Static: exists for the whole program

Common bug

Returning the address of a local (automatic) variable: the storage is gone once the function returns.

static: two very different uses

```
// (1) inside a function: value survives across calls  
void poll(void) {  
    static uint32_t call_count = 0;  
    call_count++;  
}
```

```
// (2) at file scope: internal linkage, invisible to other .c files  
static void helper(void) { /* ... */ }  
static int driver_state;
```

- Same keyword, unrelated effects – know which one you mean

volatile

```
#define GPIOA_ODR (*(volatile uint32_t *)0x48000014)
```

```
while ((SR & READY_FLAG) == 0) {  
    // wait  
}
```

- Tells the compiler: *this value can change outside the program's control*
- Without it, the compiler may cache the value in a register and the loop never sees the change
- Needed for: memory-mapped registers, variables shared with an interrupt handler
- Not a substitute for proper synchronization – more on this later in the course

Address-of and dereference

```
int x = 42;  
int *p = &x;  
*p = 7;  
  
int **pp = &p;
```

Address	Name	Value
0x2000000	x	42 → 7
0x2000004	p	0x2000000
0x2000008	pp	0x2000004

- `&x` – "address of x" – evaluates to 0x2000000
- `*p` – "value pointed to by p" – reads/writes x itself
- `*` plays two roles: in a *declaration* it marks a pointer type, in an *expression* it dereferences
- NULL means "points to nothing" – always check before dereferencing untrusted pointers

const and pointers

```
const int a = 5;           // a cannot change
```

```
int b = 10;
```

```
const int *p1 = &b;        // pointee is const: *p1 = 1; -> error
```

```
int * const p2 = &b;        // pointer is const: p2 = &a; -> error
```

```
const int * const p3 = &b;  // both are const
```

- Read the declaration right-to-left from the variable name
- const close to the type $\Rightarrow$ what it points to is protected
- const close to the name $\Rightarrow$ the pointer itself is protected

Where variables live (1/2)

Stack – local variables, call frames
Heap – malloc/free
.bss – zero-initialized globals/statics
.data – initialized globals/statics
.text / .rodata – code, constants

```

int g_uninit;           // .bss  -- global, no initializer
int g_init = 5;         // .data  -- global, non-zero initializer
const char *msg = "hi"; // "hi" itself sits in .rodata; msg (the pointer) in .data

void foo(void) {
    int local = 1;       // stack  -- automatic lifetime
    static int persist = 0; // .bss  -- static lifetime, but still block scope
}

```

- This is exactly **lifetime**, seen earlier, made concrete: automatic $\Rightarrow$ stack, static $\Rightarrow$.data/.bss

Where variables live (2/2)

Declaration	Scope	Lifetime	Segment
<code>int g; (file scope)</code>	file	static	.bss
<code>int g = 5; (file scope)</code>	file	static	.data
<code>static int g; (file scope)</code>	file, internal	static	.bss
<code>int x; (in a function)</code>	block	automatic	stack
<code>static int x; (in a function)</code>	block	static	.data/.bss
<code>"a string literal"</code>	–	static	.rodata
<code>malloc(...)</code> result	–	manual	heap

- **Scope** (who can name the variable) and **segment** (where its bytes live) are independent: `static int x;` above is block-scoped but not on the stack
- Stack grows/shrinks automatically with function calls
- Heap grows only on explicit request (`malloc`) and never shrinks back on its own

Flash vs. RAM on a microcontroller

- **Flash** (non-volatile): `.text`, `.rodata`, and the *initial values* of `.data`
- **RAM** (volatile, small): `.data`, `.bss`, stack, heap
- At startup, the runtime copies `.data` init values from flash to RAM and zeroes `.bss` – before `main()` runs

Why this matters here

RAM is tiny compared to a desktop. Every global, every stack frame, every `malloc` competes for the same few KB.

Passing parameters: value vs. pointer

```
void by_value(int x)    { x = 99; }           // caller's variable untouched  
void by_pointer(int *x) { *x = 99; }         // caller's variable modified
```

```
int v = 1;  
by_value(v);           // v is still 1  
by_pointer(&v);        // v is now 99
```

- C only has pass-by-value – a pointer is how you simulate pass-by-reference
- **Why:** each parameter is a fresh copy pushed onto the callee's stack frame – `by_value` only ever touches its own copy of `x`
- `by_pointer` receives a copy too – but what's copied is an *address*, which still lets it reach back into the caller's stack frame
- This is how a function reports more than one result, or modifies caller state

Arrays and pointer decay

```
int values[5] = {1, 2, 3, 4, 5};  
int *p = values;           // array decays to a pointer to its first element
```

```
values[2]; == *(values + 2); // strictly equivalent
```

```
sizeof(values); // 5 * sizeof(int) -- size of the whole array  
sizeof(p);      // size of a pointer -- NOT the array size!
```

- Classic bug: `sizeof` works on the array only where the array type is still visible (not once passed to a function – it becomes a pointer there)

Strings: char arrays

```
char name[6] = "hello";           // {'h','e','l','l','o','\0'}  
char *msg = "hello";             // pointer to a string literal (read-only!)  
  
strlen(name);                    // 5, does not count '\0'  
strcpy(dst, src);  
strncpy(dst, src, sizeof(dst)); // safer: bounded
```

- A C string is just bytes terminated by `'\0'` – nothing more
- Never write to a string literal (`char *msg = "..."`); it is undefined behavior
- Always size buffers with the terminator in mind

Formatted output: printf & snprintf

```
printf("count = %d, temp = %.1f\n", count, temp);
```

```
char line[32];
```

```
int n = snprintf(line, sizeof(line), "x=%d y=%d", x, y);
```

// n = number of chars that WOULD have been written -- may exceed the buffer!

- printf needs somewhere to send its bytes – on a desktop that plumbing already exists
- snprintf: same formatting, writes into a buffer instead, always bounded by `sizeof(buffer)`
- Prefer snprintf/strncpy-style bounded calls over sprintf/strcpy – the size limit is not optional in embedded code

Making printf work on the Nucleo

- printf is not magic: the C library calls a low-level output hook for every byte it writes – by default that hook goes nowhere on a bare microcontroller
- Newlib (the C library used here) calls `_write()`; STM32Cube's older syscalls stub instead calls `__io_putchar()` once per character
- Redirect either one to UART and printf works over the ST-Link virtual COM port

```
int __io_putchar(int ch) {  
    HAL_UART_Transmit(&huart2, (uint8_t *)&ch, 1, HAL_MAX_DELAY);  
    return ch;  
}  
  
int _write(int file, char *ptr, int len) {  
    HAL_UART_Transmit(&huart2, (uint8_t *)ptr, len, HAL_MAX_DELAY);  
    return len;  
}
```

struct

```
typedef struct {  
    uint16_t x;  
    uint16_t y;  
    uint8_t  pressed;  
} button_event_t;  
  
button_event_t evt = {.x = 10, .y = 20, .pressed = 1};  
evt.x = 15;  
  
button_event_t *pevt = &evt;  
pevt->x = 20;           // shorthand for (*pevt).x
```

enum

```
typedef enum {  
    STATE_IDLE = 0,  
    STATE_RUNNING,  
    STATE_ERROR  
} state_t;  
  
state_t state = STATE_IDLE;  
  
switch (state) {  
    case STATE_IDLE:    /* ... */ break;  
    case STATE_RUNNING: /* ... */ break;  
    case STATE_ERROR:   /* ... */ break;  
}
```

- Named integer constants – far more readable than magic numbers 0, 1, 2...
- Underlying type is just an `int` – no memory saved, only clarity gained
- Most compilers can warn when a `switch` over an `enum` forgets a case

union: shared storage

```
typedef union {  
    uint32_t raw;  
    uint8_t  bytes[4];  
} reg32_t;
```

```
reg32_t r;  
r.raw = 0x12345678;
```

```
// r.bytes[0] == 0x78, r.bytes[1] == 0x56, ... (little-endian on Cortex-M)
```

- All members start at the *same* address – they overlap in memory
- `sizeof(union)` = size of its largest member
- Only one member is "meaningful" at a time – writing one and reading another reinterprets the same bytes

union: a typical embedded use

```
typedef union {  
    struct {  
        uint8_t  addr;  
        uint8_t  cmd;  
        uint16_t value;  
    } fields;  
    uint8_t raw[4];  
} frame_t;  
  
frame_t f;  
f.fields.addr = 0x10;  
f.fields.cmd  = 0x02;  
f.fields.value = 1200;  
uart_send(f.raw, sizeof(f.raw)); // send the same data as a raw byte buffer
```

- Write structured fields, send/receive as raw bytes – no manual packing code
- Struct member order and padding are compiler/architecture-dependent – don't rely on this layout across different toolchains

malloc – know it, mostly avoid it here

```
int *buf = malloc(n * sizeof(int));  
if (buf == NULL) { /* allocation failed */ }  
// ... use buf ...  
free(buf);
```

- Allocates from the heap at run time; you decide when to free it
- In constrained / real-time embedded systems: fragmentation, no guaranteed timing, no easy recovery from failure
- $\Rightarrow$ generally avoided or tightly controlled on bare metal
- You will meet it again soon: FreeRTOS's default heap is dynamically allocated

#define: constants and macros

```
#define MAX_RETRIES 3
```

```
#define SQUARE(x) ((x) * (x))
```

```
SQUARE(a + b);           // expands to ((a + b) * (a + b)) -- parentheses matter!  
SQUARE(i++);             // expands to ((i++) * (i++))      -- side effect used twice
```

- Pure text substitution, before compilation even starts
- Always parenthesize the whole macro *and* every parameter
- Never pass an expression with a side effect to a macro

Conditional compilation and includes

```
#ifdef DEBUG  
    log_printf("state = %d\n", state);  
#endif
```

```
#if TARGET_BOARD == BOARD_NUCLEO  
    #include "nucleo_pins.h"  
#else  
    #include "generic_pins.h"  
#endif
```

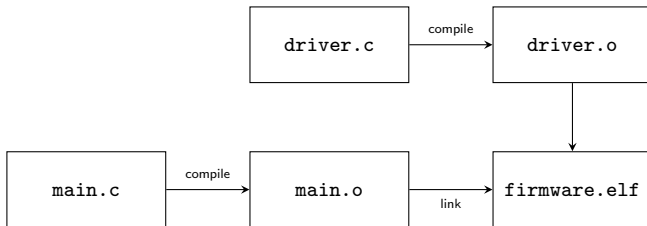
- `#include "file.h"` – local file, searched near the source first
- `#include <file.h>` – system / library header

Include guards

```
#ifndef DRIVER_LED_H  
#define DRIVER_LED_H  
  
// header content  
  
#endif // DRIVER_LED_H
```

- A header can be #included from several .c files – and indirectly, more than once
- Without a guard: redefinition errors as soon as it's included twice in one translation unit
- Equivalent modern alternative: #pragma once (not standard C, but supported everywhere we use)

From source to executable



- Each `.c` file is compiled *independently* into an object file (`.o`) – it knows nothing about the others
- The **linker** then resolves references between object files into one executable

Live demo: linker errors

```
// file a.c
void do_thing(void);
int main(void) { do_thing(); return 0; }
// -> undefined reference to `do_thing' (declared, never defined)
```

```
// file b.c and file c.c
int counter; // defined in TWO .c files
// -> multiple definition of `counter'
```

- We will trigger both errors live and read the message together
- Linker errors talk about *symbols*, not lines of source – learn to read them

.h vs. .c: declaration vs. definition

```
// led.h (declaration -- "this exists, here is its signature")  
void led_set(uint8_t id, uint8_t on);  
extern uint8_t led_count;  
  
// led.c (definition -- "here is the actual code / storage")  
uint8_t led_count = 3;  
void led_set(uint8_t id, uint8_t on) { /* ... */ }
```

- A header describes an interface; it should not itself define functions or reserve storage for variables
- `extern` tells the compiler "this global is defined elsewhere" – needed to share a variable across `.c` files

Take away

- Nothing new today – just making sure it's all fresh again
- Weak spots showed up in the quiz? Come back to them yourself before session 2
- Next session: separating BSP from application code, a layered driver model, and enough Git to pull the "bidule" project and follow along